

INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY

INTRO TO CRITICAL INFRASTRUCTURE FOR IT LEARNERS

LAB 02 — PLC PROGRAMMING LANGUAGES

Student Edition

Authorized ICDFA training environments only
icdfa.edu.ng

Lab 02: PLC Programming Languages

Course: Intro to Critical Infrastructure for IT Learners | **Estimated Duration:** 2–3 hours

Submission deadline: As published in ICDFA E-Campus.

Authorization & Safety: Perform all activities only on the ICDFA-provided virtual machines, packet captures and isolated training networks. Do not bridge these VMs to production networks, scan external systems, or reuse credentials observed in training data.

Learning Outcomes

- Compare the PLC programming languages presented in the course.
- Interpret a motor start/stop latching circuit in Ladder Diagram.
- Recognize equivalent motor-control logic represented using Function Block Diagram, Structured Text, Instruction List and Sequential Function Chart.
- Explain how tags and Boolean data types support PLC logic.

Laboratory Environment

System	Expected Address	Account	Password
PLC VM	192.168.1.10	plc	password

Note: If the instructor-provided VM image uses updated addressing or credentials, use the values published in E-Campus. Do not alter addressing unless instructed.

Pre-Lab Checklist

- Read the complete lab before beginning.
- Confirm you can access the required VM(s) and that virtualization support is enabled.
- Use the instructor-specified isolated/host-only network.
- Create or restore the clean starting snapshot for this lab.
- Prepare a report document with your name, registration number, course, lab number and date.
- Capture evidence as you progress rather than repeating the lab at the end.

Procedure

Step 1. Start from the clean ICDFA PLC VM snapshot and sign in to the PLC training environment.

Step 2. Launch OpenPLC Editor and open the Ladder Diagram example from the PLC Programming Language Examples directory.

Step 3. Open Res0 > instance0 (Motor_Control). Identify the normally open Start contact, normally closed Stop contact, MotorOutput latch contact and MotorOutput coil.

Step 4. Open the tag/variable list. Confirm that the motor-control tags use Boolean values and record the purpose of each tag in your worksheet.

Step 5. Return to File > Open and load the Function Block Diagram example. Open Motor_Control_FBD and identify the block that maintains the motor output until a stop condition occurs.

Step 6. Open the Structured Text example and inspect MotorControlST. Describe how the same start/stop behavior is represented using text rather than graphical contacts and coils.

Step 7. Open the Instruction List example and inspect MotorControl. Record the main visual difference between this low-level textual representation and Structured Text.

Step 8. Open the Sequential Function Chart example and inspect MotorControl_SFC. Identify how steps/transitions express process sequence.

Step 9. Complete the language comparison matrix in the lab report for all five representations.

Step 10. Capture one evidence screenshot for each programming representation, ensuring the program name is visible.

1. Ladder Diagram (Motor_Control)

Screenshot Available in Screenshots Folder

Identified Components:

- **Stop Contact:** Normally Closed (NC) contact shown first in the rung.
- **Start Contact:** Normally Open (NO) contact.
- **MotorOutput Latch Contact:** Parallel NO contact used for sealing/holding the circuit.
- **MotorOutput Coil:** Output coil at the end of the rung.

Logic Description: When **Start** is pressed, the **MotorOutput** coil energizes. The parallel **MotorOutput** contact creates a latch so the motor remains ON after the Start button is released. Pressing **Stop** opens the circuit and de-energizes the motor.

2. Function Block Diagram (Motor_Control_FBD)

Screenshot Available in Screenshots Folder

Identified Blocks:

- **NOT Block:** Inverts the Stop signal.
- **OR Block:** Combines Start and MotorOutput feedback signals.
- **AND Block:** Combines the OR output with the inverted Stop condition.
- **MotorOutput Variable:** Final motor output.

Motor Latch Function: The **OR block** receives both **Start** and **MotorOutput**. This feedback path keeps MotorOutput active after Start is released, providing the latching function.

Equivalent Logic:

Plain Text

MotorOutput = (Start OR MotorOutput) AND (NOT Stop)

Show more lines

3. Instruction List (Motor_Control_IL)

Screenshot Available in Screenshots Folder

Program Code Observed:

Plain Text

LD Start

OR MotorOutput

ANDN Stop

ST MotorOutput

Show more lines

Explanation:

- LD Start loads the Start signal.
- OR MotorOutput creates the latch condition.
- ANDN Stop requires Stop to remain OFF.
- ST MotorOutput stores the result in the motor output.

Main Visual Difference from Structured Text: Instruction List uses short mnemonic instructions executed sequentially, whereas Structured Text uses more readable high-level statements such as IF-THEN logic.

Tags and Data Types

Tag	Type	Purpose
Stop	BOOL	Stop pushbutton input
Start	BOOL	Start pushbutton input
MotorOutput	BOOL	Motor output and latch status

Observation: All variables use the **Boolean (BOOL)** data type, meaning they can only be TRUE (1) or FALSE (0).

4. Structured Text (Motor_Control_ST)

[Screenshot Available in Screenshots Folder](#)

The program logic is Smalltalk

MotorOutput := (NOT Stop) AND (Start OR MotorOutput);

Show more lines

Description

This single statement implements the same motor start/stop seal-in circuit:

- NOT Stop ensures the motor can only run when the Stop button is not active.
- (Start OR MotorOutput) creates the latching action.
- MotorOutput maintains itself ON after Start is released.
- Pressing Stop makes the expression FALSE and turns the motor OFF.

Observation: Structured Text expresses the control logic using Boolean operators rather than graphical contacts and coils.

5. Sequential Function Chart (Motor_Control_SFC)

From your screenshot, the variables are:

Variable	Type	Purpose
Start	BOOL	Start transition input
Stop	BOOL	Stop transition input
MotorOutput	BOOL	True while Running step is active

Description

Sequential Function Chart represents control as:

Step 1: Stopped ↓ (Start transition) **Step 2: Running** ↓ (Stop transition) **Step 1: Stopped**

The process moves between operating states through transitions rather than evaluating a continuous logic rung.

Observation: SFC is best suited for operations that have defined states and sequences.

Complete Motor Control Comparison Matrix

Language	Representation	Strengths	Limitations	Industrial Use Case
Ladder Diagram (LD)	Contacts and coils	Very easy for electricians and maintenance technicians	Large projects can become difficult to manage	Motor starters, conveyor systems
Function Block Diagram (FBD)	Connected logic blocks	Clear signal flow and modular design	Complex systems may appear cluttered	Process control and instrumentation
Structured Text (ST)	High-level text code	Excellent for mathematical and logical operations	Requires programming knowledge	Advanced automation systems
Instruction List (IL)	Low-level mnemonic instructions	Compact representation	Difficult to read and maintain	Legacy PLC systems
Sequential Function Chart (SFC)	Steps and transitions	Ideal for sequential operations	Less effective for simple control tasks	Batch processing and machine sequencing

Short Explanation:

The motor start/stop requirement is implemented differently in each PLC programming language while maintaining the same control objective. Ladder Diagram uses contacts and coils to create a latching circuit. Function Block Diagram uses interconnected logic blocks, including feedback from MotorOutput to maintain the latch. Structured Text expresses the same logic in a Boolean expression. Instruction List uses mnemonic commands such as LD, OR, ANDN, and ST to achieve the same behavior. Sequential Function Chart represents the process as states and transitions between stopped and running conditions. Although the representation changes, the Boolean tags Start, Stop, and MotorOutput retain the same meaning and produce identical motor-control behavior.

Knowledge Check

1. Why is Ladder Diagram intuitive for technicians familiar with relay logic?

Response: Ladder Diagram closely resembles traditional electrical relay control circuits, using contacts, coils, and rungs that technicians already understand, making it easy to read, troubleshoot, and maintain PLC programs.

2. What is one advantage of Structured Text for complex calculations or conditions?

Response: Structured Text allows complex mathematical calculations, nested conditions, loops, and data manipulations to be written efficiently in a clear textual format, reducing program complexity compared to graphical languages.

3. When might a Sequential Function Chart be more appropriate than a single ladder rung?

Response: Sequential Function Chart is more appropriate when a process involves multiple steps, states, or sequences, such as automated manufacturing operations, batch processing, or machine startup and shutdown procedures.

4. Why must the meaning of tags remain consistent when the representation/language changes?

Response: Tags represent the same inputs, outputs, and internal variables regardless of the PLC programming language used. Keeping tag meanings consistent ensures the control logic performs the same function and avoids programming errors when converting between different representations.